Scale Models and Logical Clocks CS 262: Distributed Computing

Pratyush Mallick | Anika Lakhani - March 8th, 2023

INTRODUCTION

In this assignment, we were simulating three different machines that operated at different speeds being able to message each other. Each message had its own internal clock that replicated a machine being able to operate at a certain speed in real life. According to this internal clock, the machine carried out commands based on the results of the random number generator, which simulated a user sending unpredictable requests to the application.

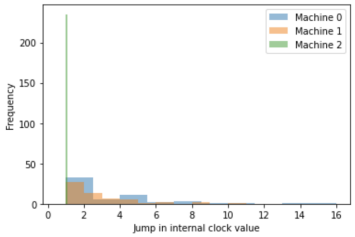
DESIGN

We based our design largely off of Professor Waldo's starter code. Our project had three virtual machines in which each machine had a "first neighbor" and a "second neighbor." If a given machine where to send a message, they could either send this message to the first neighbor, the second neighbor, or both neighbors. We recorded timing data for this message sending. Received messages stacked up in a queue that could be accessed by the receiving node according to the specifications. Here we've included some sample data to show you what the collected data for Machine 0 looks like:

Sender Machine, $(1, 2)$	Logical Clock Time	Global Clock Time
2	1	1677965275.495232
1	2	1677965275.4977531
1	3	1677965275.4980228
2	4	1677965276.493224

At a high level, our application instantiates three separate Processes (using Python's multiprocessing package), each corresponding to one of the virtual machines.
The machines are identical, except for having different identifiers, and execute the following functionality:

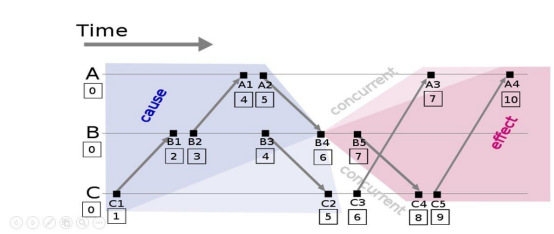
- Maintain a queue of pending events.
- Maintain a clock (int) that stores the current internal timestep of the machine.
- Write events to a machine-specific log as specified below.
- Bind to a socket, reserved for this machine, and listen for incoming messages, which are added to the queue.
- Execute at most a pre-specified number of the following instructions per second. Each instruction is executed and then is written to a log. If the queue is empty, o



- One of the remaining instructions is chosen (with pre-specified probabilities). After each instruction, the internal clock is incremented by 1.
    - Pop an event (i.e. incoming message) off the queue.
    - Send a message (over appropriate socket) to one of the other two machines.
    - Send a message to both of the other two machines.
    - Internal event.

To maintain the internal logical clocks, we used Lamport's timestamp algorithm. We chose this algorithm because it is simple (each machine just maintains a single clock value), but still allows us to infer some information about the ordering of events from the logs. We update the internal clocks according to the following rules: The clock is incremented before executing any instruction (listed above). Any messages sent by a machine include the clock value of that machine.

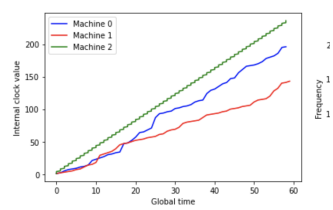
Having received a message from another machine, which includes a clock value, the clock value on the current machine is updated to be the maximum of itself and the received clock value.



We also decided to run the virtual machine as a separate Process, using Python's multiprocessing package. To model the fact that separate machines do not share memory, we used Processes instead of Threads (which do share memory).

As an example, we ran the scale model with the following settings:

- Machine speeds 1, 1, 4
- Duration of simulation was 60 seconds
- Probability of internal event was 0.4.



VARIATION

The scale model was executed for different settings of the following parameters:

- Variation in the clock cycles
- Probability of an event being internal

We found that:

- Smaller variation in clock cycles naturally resulted in less drift and smaller jumps in clock values. As expected, queue sizes will still be large if there is also a smaller probability of an event being internal.
- Smaller probability of an event being internal significantly exacerbated the effects described above. Namely, we observed significantly larger jumps in values of logical clock values, larger queue sizes and larger drift, for slower machines. This makes sense; smaller probability of internal events means more messages are sent.

UNIT TESTING

Because of our design choices, our implementation of clocks does not contain any input or output methods but rather runs several workers through different processes and threads that don't necessarily return any values. These processes modify data structures and print to the LOG txt files. We completed unit testing by using assertions within each method, ensuring that messages were sent and received in a valid format.

```
def assertConfig(config):
  '''Check that an input config is valid.'''

  # Validate host address using a regex.
  # https://www.geeksforgeeks.org/python-
  # program-to-validate-an-ip-address/.
  ipRegex = "^((25[0-5]|2[0-4][0-9]|1[0-9]
      [0-9][1-9]?[0-9])\.){3}"
  ipRegex += "(25[0-5]|2[0-4][0-9]|1[0-9]
      [0-9]|[1-9]?[0-9])$"
  assert(type(config[0]) == str)
  assert(re.match(ipRegex, config[0]))

  # Check that port is in valid range, and
  # that tick size is at least 1.
  assert(type(config[1]) == int)
  assert(config[1] >= 0 and
    config[1] <= 65535)
  assert(type(config[1]) == int)
  assert(config[2] >= 1)
```